Atelier1 : JS+NodeJS+MongoDB

1. Docker Compose

```yaml
# docker-compose.yml
version: '3.8'
services:
  mongodb:
    image: mongo:latest
    container_name: mongodb
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: password
    volumes:
      - mongodb_data:/data/db

  mongo-express:
    image: mongo-express:latest
    container_name: mongo-express
    ports:
      - "8081:8081"
    environment:
      ME_CONFIG_MONGODB_ADMINUSERNAME: admin
      ME_CONFIG_MONGODB_ADMINPASSWORD: password
      ME_CONFIG_MONGODB_SERVER: mongodb
      ME_CONFIG_BASICAUTH_USERNAME: admin
      ME_CONFIG_BASICAUTH_PASSWORD: password

volumes:
  mongodb_data:
```

Lancez les services :

```bash
docker-compose up -d
```

2. Serveur Express Minimal

package.json

```json
```

```json
{
  "name": "product-api",
  "type": "module",
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.5.0",
    "multer": "^1.4.5"
  }
}
```

server.js

```javascript
import express from 'express';
import mongoose from 'mongoose';
import multer from 'multer';
import path from 'path';

const app = express();
const PORT = 3000;

// Middleware
app.use(express.json());
app.use('/uploads', express.static('uploads'));

// Connexion MongoDB
mongoose.connect('mongodb://admin:password@localhost:27017', {
    dbName: 'products_db'
});

// Schéma Produit
const productSchema = new mongoose.Schema({
    id: { type: String, required: true, unique: true },
    nom: { type: String, required: true },
    prix: { type: Number, required: true },
    image: String
});

const Product = mongoose.model('Product', productSchema);

// Configuration Multer
const storage = multer.diskStorage({
    destination: 'uploads/',
    filename: (req, file, cb) => {
```

```
        cb(null, Date.now() + path.extname(file.originalname));
    }
});

const upload = multer({ storage });

// Routes
app.post('/api/products', upload.single('image'), async (req, res) => {
    try {
        const { id, nom, prix } = req.body;

        const product = new Product({
            id,
            nom,
            prix: parseFloat(prix),
            image: req.file?.filename
        });

        await product.save();
        res.json({ message: 'Produit ajouté', product });
    } catch (error) {
        res.status(400).json({ error: error.message });
    }
});

app.get('/api/products', async (req, res) => {
    const products = await Product.find();
    res.json(products);
});

app.listen(PORT, () => {
    console.log(`Serveur: http://localhost:${PORT}`);
    console.log(`Mongo Express: http://localhost:8081`);
});
```

3. Client HTML Minimal

index.html

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Ajouter Produit</title>
</head>
<body>
    <form id="productForm">
        <input type="text" name="id" placeholder="ID" required>
        <input type="text" name="nom" placeholder="Nom" required>
        <input type="number" name="prix" placeholder="Prix" step="0.01" required>
        <input type="file" name="image" accept="image/*" required>
        <button type="submit">Ajouter</button>
    </form>

    <div id="message"></div>

    <script>
        document.getElementById('productForm').addEventListener('submit', async (e
) => {
            e.preventDefault();

            const formData = new FormData(e.target);

            const response = await fetch('http://localhost:3000/api/products', {
                method: 'POST',
                body: formData
            });

            const result = await response.json();
            document.getElementById('message').textContent =
                response.ok ? 'Produit ajouté!' : 'Erreur: ' + result.error;
        });
    </script>
</body>
</html>
```

4. Installation et Lancement

```
# 1. Lancer MongoDB et l'interface
docker-compose up -d

# 2. Installer les dépendances Node.js
npm install

# 3. Démarrer le serveur
node server.js
```

```
# 4. Ouvrir dans le navigateur
# http://localhost:3000 (serveur API)
# http://localhost:8081 (interface MongoDB - admin/password)
```

5. Structure finale

```
.
├── docker-compose.yml
├── server.js
├── package.json
├── index.html
└── uploads/ (créé automatiquement)
```

Accès aux interfaces :

- **API Server** : http://localhost:3000
- **MongoDB Express** : http://localhost:8081 (admin/password)

Créer un endpoint Express pour servir des images

Maintenant, on va voir comment créer un endpoint minimaliste pour servir des images depuis un dossier local avec Express.

Configuration Express

```javascript
// server.js
const express = require('express');
const path = require('path');

const app = express();

// Servir les images statiquement
app.use('/images', express.static(path.join(__dirname, 'uploads')));

// Endpoint alternatif avec contrôle
app.get('/image/:nom', (req, res) => {
    const imagePath = path.join(__dirname, 'uploads', req.params.nom);
    res.sendFile(imagePath, (err) => {
```

```
        if (err) res.status(404).send('Image non trouvée');
    });
});

app.listen(3000, () => console.log('🖼️  Serveur images: http://localhost:3000'));
```

Structure du projet

```
project/
 ├─ server.js
 └─ uploads/
      └─ photo1.jpg
```

Installation et lancement

```
# Initialisation
npm init -y
npm install express

# Lancement
node server.js
```

Utilisation

Méthode 1 - Service statique :

```
http://localhost:3000/images/photo1.jpg
```

Méthode 2 - Endpoint personnalisé :

```
http://localhost:3000/image/photo1.jpg
```

Résultat

- Serveur Express minimal (10 lignes)
- Service d'images automatique
- Gestion d'erreurs 404
- URL propres et simples

Testez dans votre navigateur :
👉 `http://localhost:3000/images/photo1.jpg`

L'image s'affiche instantanément ! 🎉